

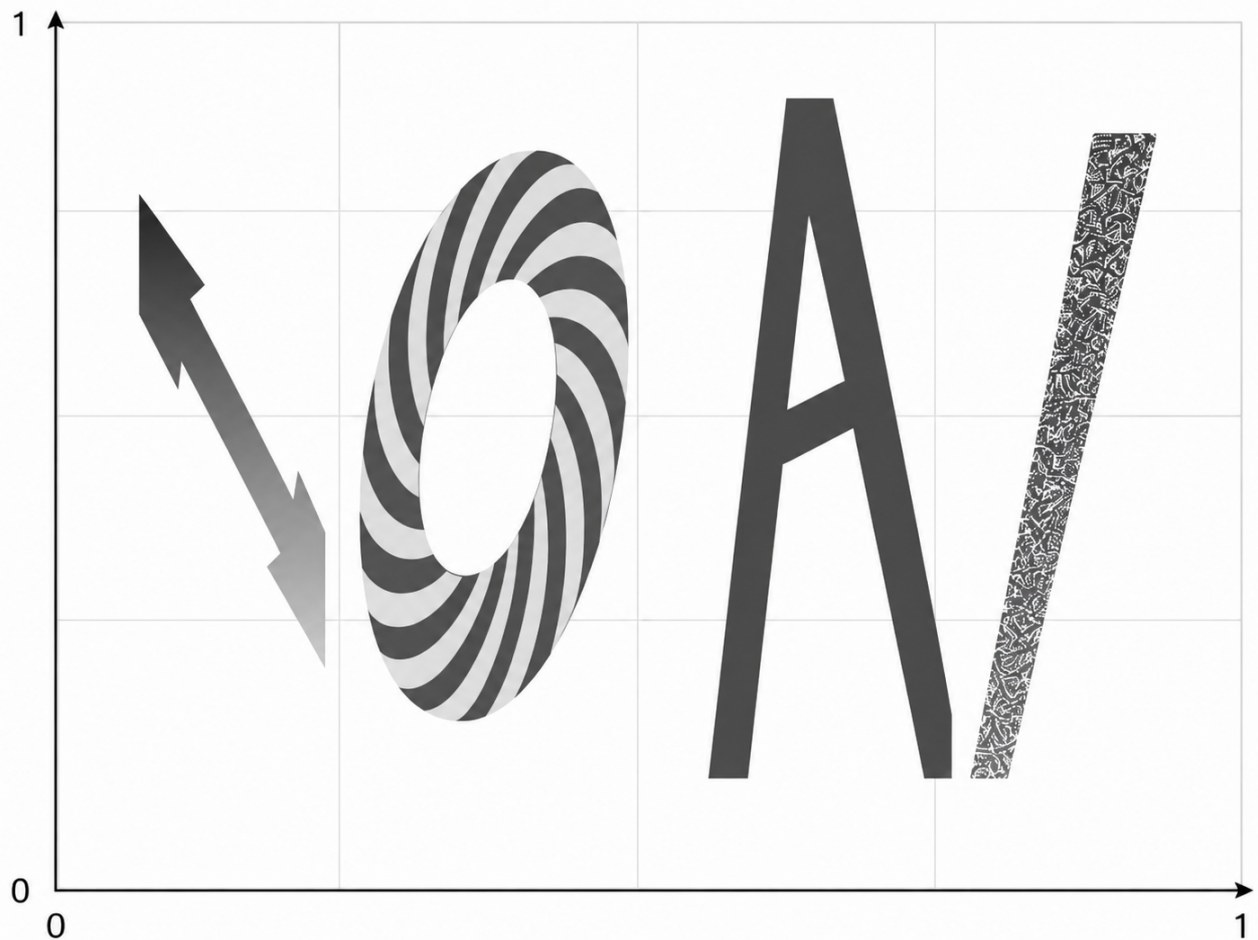
IOAIフィールド

- 制限時間: 5 minutes
- ストレージ: 5 GB
- 提出物のサイズ: `solution.ipynb`, `custom_model.py` を合わせて ≤ 1 MB
- 事前学習済みモデル: なし — ゼロから学習し、採点時にはインターネットを使用できません
- ベースラインスコア: 31.2187

課題

Astana市長は、様式化されたIOAIのロゴで市を飾りたいと考えています。統計学者である市長は、ロゴを含むあらゆるものを空間関数 $F(x, y, \overline{W})$ として捉えます。ここで、 $x, y \in [0, 1]$ は2D平面上の座標を表し、 $\overline{W}$ は文字の色や角度などの様式的属性を定義する隠れパラメータの集合です。

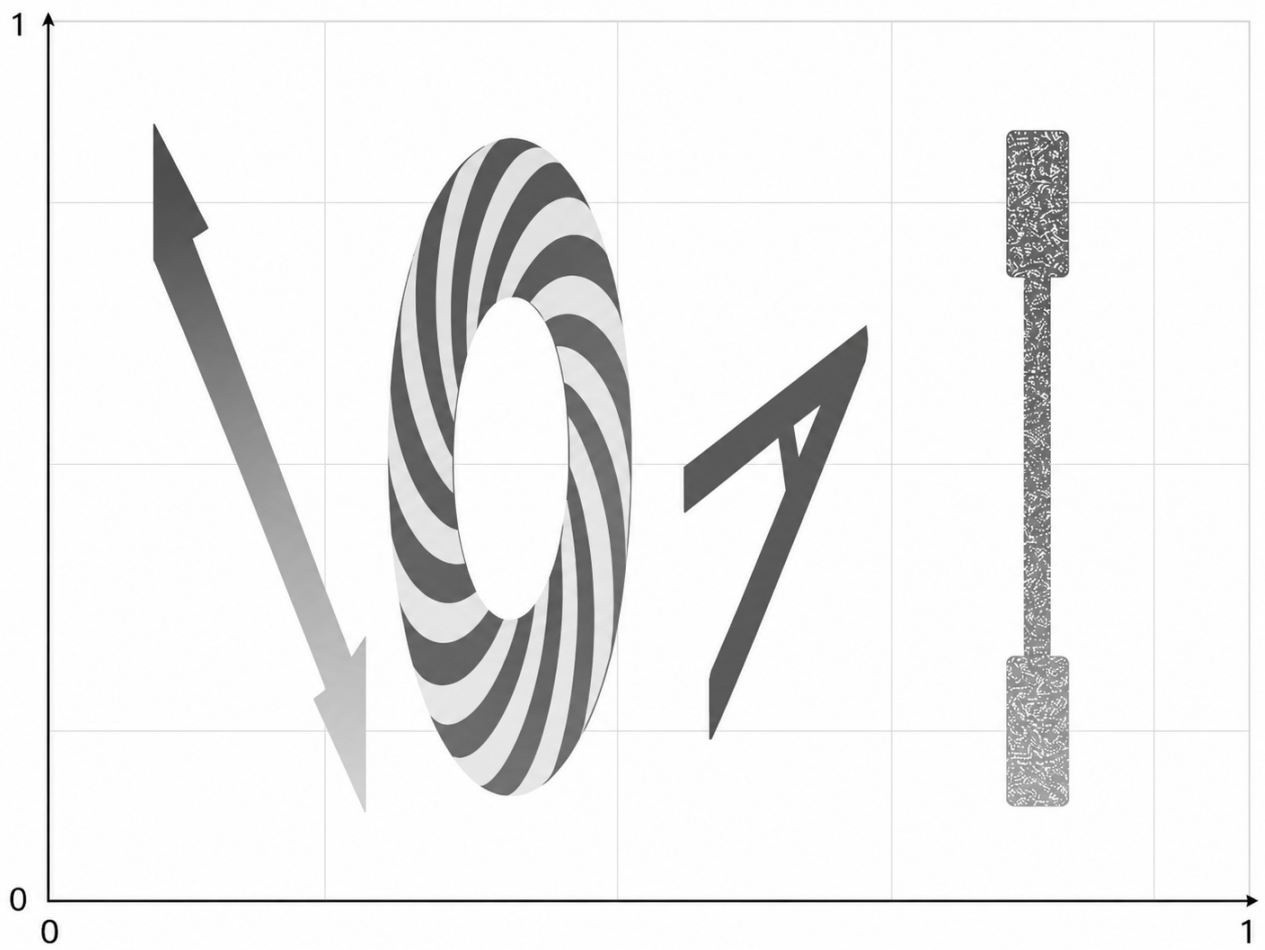
F は明示的な数式で表すには複雑すぎるため、これを近似するニューラルネットワークを学習させることが課題です。ネットワークは任意の座標ペア (x, y) に対する **IOAIフィールド** の値を出力し、平面全体にわたるロゴの完全なヒートマップ可視化を生成します。以下は、特定の隠れパラメータ $\overline{W}$ における F のヒートマップ可視化の例です。

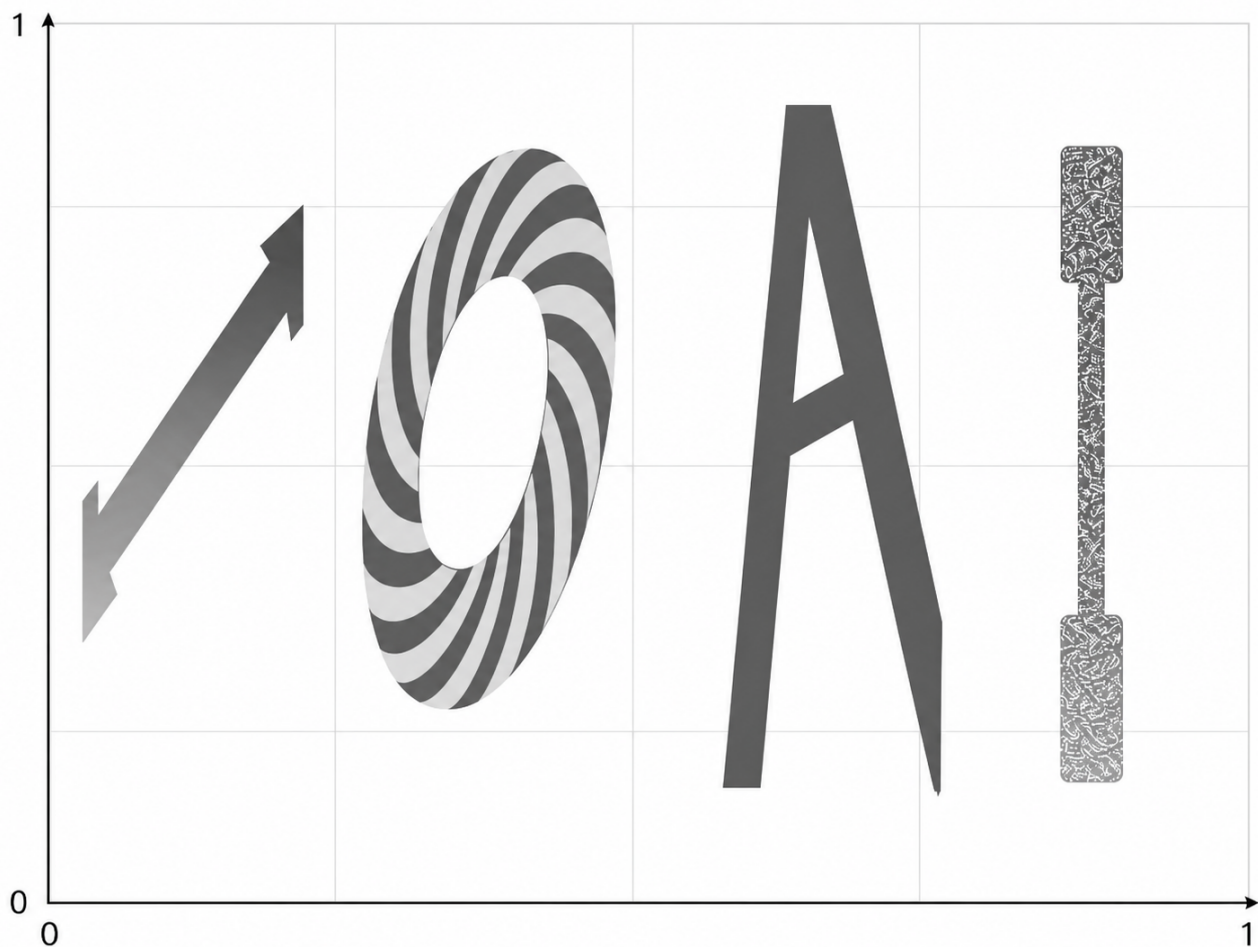


IOAIフィールドは何で構成されているでしょうか？4つの文字と背景です。

- 最初の I の文字内の値は非常に大きく ($1e+10$ 以上)、線形勾配を持ちます
- 文字 O 内の値はらせんパターンを示します
- 文字 A 内の値は常に -1 です
- 最後の I の文字内の値は、同じ点で複数回評価した場合でも、範囲 $[-2026, 2026]$ からのランダムな値でなければなりません
- 文字の外側の値は常にゼロです

関数には隠れパラメータ $\overline{W}$ があり、これは文字のスケールと傾き、および最初の I の文字内の値の範囲に影響します。ただし、文字同士が交差することはありません。以下は、異なる $\overline{W}$ におけるIOAIフィールドの外観を示すいくつかの例です。





与えられるもの:

この問題にはデータセットが一切含まれていません。代わりに、`data/train_config/field_config.json`にあるJSON設定ファイルによって設定される生成関数が与えられます。

テスト用の設定は非公開ですが、同様の性質を持ちます。望むだけ多くのデータを使用して、与えられた生成器に適合させることが課題です。「訓練」分布と「テスト」分布は同じ生成器から生成されますが、どの点 (x_i, y_i) で評価されるかは分かりません。

提出物は以下で構成されなければなりません。

- `custom_model.py` として保存された学習用モデルクラス。このモデルは `torch.nn.Module` クラスを継承し、`torch` の `import` のみを使用しなければなりません。また、`solution.ipynb notebook` で使用される `CustomModel` クラスを含まなければなりません。
- `model.pt` の重みを生成する `solution.ipynb notebook`

採点

各領域について、最小スコアは0、最大スコアは1です。最終スコアは5つすべての領域（各文字について4つ、および背景）で平均され、100倍されます。パラメータペナルティがあります。

モデルのパラメータ数が20260より多い場合、スコアは半分になります。

パラメータ数は `sum(p.numel() for p in model.parameters())` によって測定されます。モデルの一部としてPyTorchの `nn.Dropout` を使用し、モデルが確率的モードでも動作することを想定しています。

標準文字の領域について

各領域 R (最初の $\mathbb{I}$ の文字、 $\circ$ 、 A 、 $Background$) について、真の値 v_i と予測値 $\hat{v}_i$ を持つ $N_R = 512$ 個のテスト点 (x_i, y_i) でモデルを評価します。主要な評価指標として、正規化された平均絶対誤差 (MAE) を使用します。MAEは次のように定義されます。

$$\text{MAE}(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

また、正規化は次のように行われます。

$$\text{score}(R) = 1 - \min \left(\frac{\text{MAE}(R)}{s_R}, 1 \right),$$

ここで、 $s_R > 0$ はスケール定数です。

最後の $\mathbb{I}$ の文字領域について

この領域では、**評価中にモデルのdropoutが有効化されます**。各テスト点 j について、次の処理を行います。

1. モデルを $K = 10$ 回実行して、 $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$ を取得します。
2. いずれかの出力が範囲 $[-2026, 2026]$ の外にある場合、 $\text{pointScore}(j) = 0$ とします。
3. それ以外の場合、 K 個の出力の標準偏差 σ_j を計算し、スコアに変換します。

$$\text{pointScore}(j) = \min \left(\frac{\sigma_j}{s_E}, 1 \right),$$

ここで、 $s_E > 0$ は固定されたスケール定数です。

領域スコアは、その領域内のすべての点についての平均です。

$$\text{score}(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} \text{pointScore}(j),$$

ここで、 $N_E = K * N_R$ です。

簡単に言えば、多様性が高いほど、この領域のスコアは大きくなります。PyTorchの `rand*` 関数および `_uniform` 関数を含め、純粋な形でrandomを使用することはできません。ランダム性は、dropoutを有効にした推論から生じなければなりません。

提出方法

1. `solution.ipynb` を開き、すべてのセルを実行します。
2. `custom_model.py` 内の `CustomModel` モデルを改善します
3. 最後のセルでモデルが `model.pt` ファイルに保存されることを確認します。
4. JupyterLabのGitタブで、`solution.ipynb` と `custom_model.py` をステージし、コメントしてコミットした後、pushします。
5. Contestページに戻り、**Submit**をクリックします。提出コメントは、前の手順のコメントと同じでなければなりません。